Design Document — AI-Powered Telecom Network Operations Assistant

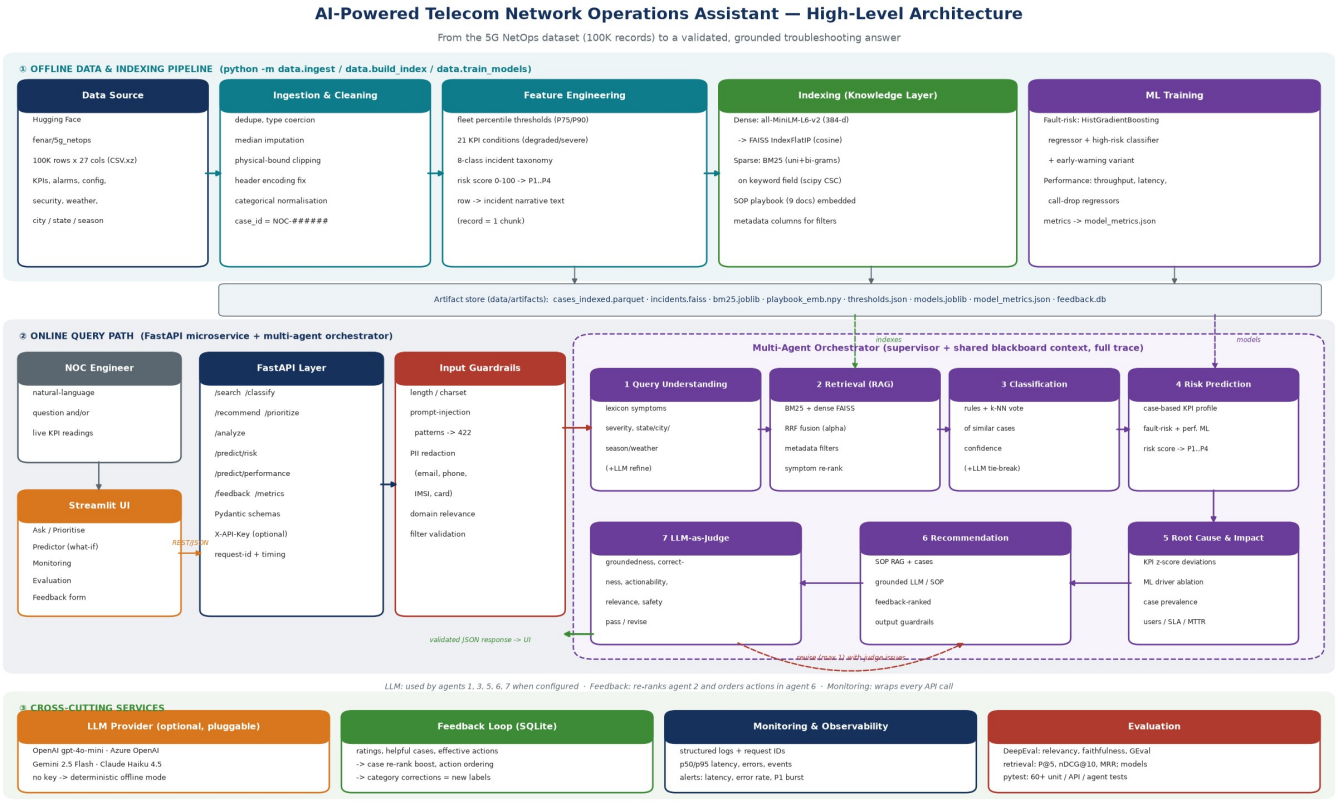


Figure 1 - High-level architecture (full size: 01_Architecture.pdf)

Project 4 · PU Bootcamp GenAI Capstone · Author: Jerom Santhiyagu (Prodapt)

1. Problem and goals

NOC engineers have to search thousands of historical network records to find similar incidents, work out the likely cause and decide on corrective actions. The goal is a microservice where an engineer types a natural-language question (optionally with live KPIs) and gets back:

- the most similar historical network cases, found with hybrid semantic + keyword search and metadata filters;
- an incident classification, a priority (P1–P4) and a 0–100 risk score;
- predicted fault risk and network performance;
- a root-cause and network-impact analysis;
- a grounded, prioritised troubleshooting plan, checked by an LLM-as-judge;
- a feedback loop that improves ranking and recommendations over time.

Design principles: **grounded** (every recommendation cites a retrieved case or an SOP), **always available** (fully functional without an LLM key, and better with one), **transparent** (an agent trace, scores and evidence come back with every answer), and **measurable** (automated retrieval, recommendation and DeepEval evaluations).

2. Requirements traceability

Requirement	Implementation	API	Tests
Basic RAG similarity search	narratives → MiniLM embeddings → FAISS; RAG over SOP playbook	`/search`, `/recommend`	test_api::test_search_modes
Hybrid search (keyword + semantic)	BM25 (scipy) + dense, Reciprocal Rank Fusion, alpha weight	`/search` `mode`, `alpha`	test_search_modes, retrieval_eval

Requirement	Implementation	API	Tests
Incident classification agent	rules + k-NN vote of similar cases (+ LLM tie-break)	`/classify`	test_classify_query_and_kpis
Metadata filtering (state, city, season, conditions)	boolean pre-filter → FAISS `IDSelectorBatch`; auto-detected from text	`filters` on every query endpoint	test_metadata_filters_are_enforced
Basic troubleshooting recommendation	SOP playbook + similar cases, feedback-ranked	`/recommend`	test_recommend_basic
Input validation guardrails	Pydantic schemas, injection / PII / domain checks, output sanitiser	all	test_guardrails.py
Incident prioritisation	calibrated risk score → P1–P4; batch ranking	`/prioritize`	test_prioritize_orders_by_priority
DeepEval recommendation evaluation	AnswerRelevancy, Faithfulness, ContextualRelevancy, 2× GEval	`evaluation/deepeval_eval.py`	test_deepeval.py
Network fault-risk prediction	HistGradientBoosting regressor + classifier + early-warning model	`/predict/risk`	test_predict_endpoints
LLM-as-judge diagnosis validation	Judge agent, 5 criteria, pass/revise, revision loop	`/analyze` → `judge`	test_agents.py
Multi-agent troubleshooting collaboration	7 agents, supervisor + blackboard, judge → recommender loop	`/analyze`	test_revision_loop_runs_when_judge_rejects
Network performance prediction	throughput / latency / call-drop regressors	`/predict/performance`	test_predict_endpoints
Root-cause & network-impact analysis	z-score deviations + ML driver ablation + case prevalence + impact model	`/analyze` → `root_cause`, `impact`	test_analyze_full
Feedback loop	SQLite store → case re-rank boost, action ordering, label corrections	`/feedback`, `/feedback/stats`	test_feedback_loop_changes_ranking
Simple front end	Streamlit, 5 tabs	`frontend/streamlit_app.py`	manual / demo

3. Architecture overview

See `01_Architecture.pdf`. The system has three layers.

1. **Offline pipeline** (`data/`): ingestion → feature engineering → indexing → model training. All outputs are versionable files in `data/artifacts`, so the online service is stateless apart from the feedback DB.
2. **Online microservice** (`app/`, `agents/`): FastAPI → guardrails → multi-agent orchestrator → JSON response.
3. **Cross-cutting**: a pluggable LLM client, the feedback loop, monitoring and evaluation.

Code is organised by responsibility (`app/` service and infrastructure, `agents/` reasoning, `data/` pipeline, `utils/` domain knowledge and features, `evaluation/`, `tests/`, `frontend/`). Each agent is a small class with a single `run(ctx)` method, so agents can be tested, swapped or reordered independently.

4. Data design

Dataset. 100,000 records × 27 columns (Hugging Face `fenar/5g_netops`): KPIs (availability, throughput, latency, loss, jitter, call drop, handover, signal, setup success), alarms, configuration changes, users, data usage, security and authentication events, weather and environment, location (city, state, zip), season, `Issue Reported` and `Fault Occurrence Rate`.

Cleaning. Dedupe, numeric coercion with median imputation, clipping to physical bounds, normalising the categorical values, and fixing the broken `Temperature (°C)` header encoding. The HF data turned out to be clean (no duplicates or NaNs), but the guards stay in place for real feeds.

Feature engineering (`utils/features.py`). The dataset has no free text, so the main design decision was how to turn numbers into the language engineers use:

- 21 **KPI conditions** (e.g. `high_latency`, `handover_failure`), each *degraded* at the fleet P75 (P25 for "low is bad" KPIs) and *severe* at P90 (P10). Thresholds come from the data, not hard-coded, and are stored in `thresholds.json`, so the same definitions apply at query time.
- An **8-class incident taxonomy** (Radio Access & Coverage, Capacity & Congestion, Transport & QoS, Mobility &

Handover, Equipment Fault & Alarms, Configuration, Security, Environmental) plus Normal. Each category is a weighted set of conditions.

- A **risk score** = 50 % fault likelihood (percentile of the fault occurrence rate) + 30 % degradation breadth

(number of degraded KPIs) + 20 % customer impact (users and availability loss). The thresholds were calibrated on the distribution to give roughly 15 % P1, 17 % P2, 28 % P3 and 40 % P4, which keeps NOC queues workable.

- An **incident narrative** per record: context (season, place, weather), observed conditions with values,

classification, then the full KPI dump.

Chunking. Each record is one atomic incident, so **one record = one chunk**. Splitting a record would break the link between symptoms and their context, and merging records would blur case identity. Two text fields are indexed: *summary* (context + conditions + classification, which fits MiniLM's 256-token window) for dense retrieval, and a *keyword field* (location, season, weather, condition labels, category) for BM25. Raw numbers are left out of BM25 on purpose: they add vocabulary noise and carry no lexical meaning.

Data findings that shaped the design.

- `Issue Reported` is about 50/50 and statistically independent of every KPI (the group means are identical), so it is

label noise and is **not** used as a target. It is only exposed as a filter.

- `Fault Occurrence Rate` is an almost deterministic function of the other KPIs ($R^2 \approx 1.0$, driven by Critical

Alarm Count r = 0.93, Authentication Failures 0.64 and Call Drop Rate 0.46). The full model is therefore used for *what-if* analysis, and an **early-warning model** without the near-coincident symptoms (critical alarms, throughput) gives an honest forecast from leading indicators ($R^2 \approx 0.64$).

- The data is synthetic (for example, about 38K distinct city names), so city filters are exact-match and states are

the main geographic dimension.

5. Retrieval and RAG strategy

Decision	Choice	Alternatives considered	Rationale / trade-off
Dense model	`all-MiniLM-L6-v2` (384-d)	bge-small-en-v1.5, all-mpnet-base-v2, TF-IDF+LSA	Best quality per unit of compute (§6): within about 1 nDCG point of bge-small at 2.2× its encoding speed; mpnet is both slower (~7×) and less accurate; small index (150 MB for 100K)
Vector store	FAISS `IndexFlatIP` (exact cosine)	HNSW, IVF-PQ, ChromaDB, managed vector DB	100K × 384 exact search takes milliseconds, has no recall loss and needs no server. HNSW/IVF or a managed DB becomes worthwhile beyond about 5M vectors (§13)
Sparse	Custom vectorised BM25 (k1 = 1.5, b = 0.75, uni+bi-grams) on a scipy CSC matrix	`rank_bm25`, Elasticsearch/OpenSearch	Scores 100K docs in a few ms; bi-grams capture "packet loss" and "call drops"; no extra infrastructure
Fusion	Reciprocal Rank Fusion (k = 60) weighted by alpha (default 0.6 semantic)	linear score blending	RRF is robust to the very different score scales of BM25 and cosine; alpha is tunable per request
Query expansion	The lexicon maps synonyms ("lag", "dropped calls") to canonical condition labels for BM25	LLM query rewriting	Deterministic, zero latency; the LLM can add conditions on top
Re-ranking	0.65 × fused rank + 0.35 × symptom overlap with the query, + feedback boost (±0.15, tanh)	cross-encoder	Structured overlap is precise for KPI symptoms and costs nothing; a cross-encoder could be added later
Metadata filtering	Boolean pre-filter mask → FAISS `IDSelectorBatch`; BM25 masked	post-filtering	Pre-filtering guarantees top-k *within* the filter (post-filtering can return 0 hits)
Auto filters	state, city, season and weather detected in the query are applied as *soft* filters and relaxed if they give 0 hits	hard filters only	Lets engineers write "call drops in Texas in winter" without using the filter UI
Top-K	5 by default (1–20)	—	Enough evidence for voting and grounding without diluting the LLM context

The **RAG grounding context** has the top-k case narratives plus the SOP entries for the predicted (and secondary) categories, retrieved from the 9-document troubleshooting playbook (`utils/telecom_kb.py`). The LLM is told to

cite case IDs and SOPs. The output guardrail then removes any citation that was not in the evidence.

Retrieval strategy results (live 100K index, 40 paraphrased labelled queries):

strategy	p@5	ndcg@10	mrr@10	symptom_recall@10	latency_ms_p50
keyword_bm25	0.745	0.826	0.750	0.825	9.400
semantic_dense	0.645	0.799	0.730	0.795	29.100
hybrid_rrf	0.650	0.802	0.747	0.796	37.200
hybrid_rrf+symptom_rerank	0.890	0.934	0.950	0.923	37.600
hybrid_alpha_0.3	0.825	0.914	0.838	0.912	36.700
hybrid_alpha_0.5	0.865	0.937	0.931	0.935	36.200
hybrid_alpha_0.7	0.885	0.932	0.944	0.921	36.000
hybrid_alpha_0.9	0.885	0.929	0.940	0.917	39.000

Reading the results.

- BM25 on its own is a strong baseline here. The query parser's lexicon expands paraphrases into the canonical condition labels, which BM25 matches exactly.
- Dense retrieval adds robustness for wording the lexicon does not cover. On its own it ranks less precisely, because narratives that share context (season, weather) look similar even when their symptoms differ.
- **Plain RRF fusion does not beat BM25 alone.** The large gain comes from the structured **symptom re-ranking** on top of the fused candidate pool (nDCG@10 about 0.83 → 0.93, MRR about 0.75 → 0.95). The fused pool gives the re-ranker a wider and more diverse set of candidates than either retriever alone.
- The alpha sweep is flat between 0.5 and 0.9. The default of 0.6 sits in that plateau.

The production default is therefore *hybrid + re-rank*. Latency is about 40 ms over 100K cases.

6. Embedding model evaluation

Four embedders were benchmarked on the same 4,000-case subset. The 40 labelled queries are phrased with **paraphrases that do not match the index vocabulary** (e.g. "packets being dropped" rather than "packet loss"), so pure keyword matching cannot trivially win. Relevance is graded as the fraction of the queried symptoms the case actually shows. The metrics are P@5 (all symptoms present), nDCG@10, MRR@10 and encoding throughput.

model	dim	docs/s	dense nDCG@10	dense P@5	dense MRR	hybrid nDCG@10	hybrid P@5
all-MiniLM-L6-v2	384	82.300	0.735	0.550	0.705	0.945	0.895
bge-small-en-v1.5	384	37.100	0.741	0.550	0.697	0.956	0.915
all-mpnet-base-v2	768	12.300	0.706	0.495	0.679	0.932	0.865
tfidf-lsa	256	20740.200	0.503	0.315	0.385	0.816	0.630

Decision. `bge-small-en-v1.5` is marginally the most accurate (hybrid nDCG@10 0.956 vs 0.945), but it encodes at less than half MiniLM's speed on CPU. That matters, because indexing 100K records already takes about 20 minutes with MiniLM. The larger `all-mpnet-base-v2` is slower *and* less accurate on these short, templated narratives. TF-IDF/LSA is roughly 250× faster but clearly worse; it is kept as the fully offline fallback. **MiniLM is the default.** bge-small is a one-line switch (`NETOPS_EMBEDDING_MODEL`) for deployments that can afford the extra indexing time. In every case the hybrid + re-rank pipeline narrows the gap between models considerably, so model choice matters less than retrieval design here.

7. LLM selection

Model	Relative cost	Latency	Fit for this task
GPT-4o-mini (default for OpenAI/Azure)	very low	low	Strong JSON mode and instruction following; enterprise route via Azure OpenAI (taught in the bootcamp)
Gemini 2.5 Flash	very low	low	Similar capability, generous free tier, native JSON output
Claude Haiku 4.5	low	low	Strong grounded reasoning and refusal behaviour
GPT-4o / Claude Sonnet / Gemini Pro	10–20× higher	higher	Better long reasoning, but our prompts are short, structured and evidence-bound, so the gain did not justify the cost

Model	Relative cost	Latency	Fit for this task
Local Llama/Mistral (Ollama)	infra only	high on CPU	Good for data residency; possible through the same client interface

Decision. A small, fast model is the default, behind a provider-agnostic `LLMClient` (OpenAI, Azure OpenAI, Gemini, Anthropic), with temperature 0.1 (0.0 for the judge), JSON-mode output and a 45 s timeout. The heavy lifting (retrieval, classification, risk and prediction) is done by deterministic components and ML models, so the LLM is used only where language helps: query refinement, classification tie-breaks, the root-cause narrative, plan writing and judging. That keeps cost at about 5 LLM calls per `/analyze` (roughly 4–6K tokens, a fraction of a cent with the mini/flash models).

Graceful degradation. With no key, or if any LLM call fails, every agent falls back to its deterministic implementation: SOP planner, heuristic judge and template root cause. The trace shows which path ran. This also makes the test suite deterministic and offline.

8. Multi-agent design

Pattern. A supervisor-orchestrator with a shared **blackboard** (`AgentContext`). Each agent reads and writes typed fields and appends a trace record (status, latency, whether the LLM was used, notes). An agent failure is caught and recorded, and the pipeline continues.

#	Agent	Responsibility	Tools / signals	LLM role
1	Query Understanding	symptoms, severity, location/season/weather, KPI-derived conditions	lexicon, US-state map, known cities	adds missed conditions (allowed keys only)
2	Retrieval	hybrid search, filters, filter relaxation, feedback boost	FAISS, BM25, feedback store	—
3	Classification	category and confidence: 60 % rule evidence + 40 % similarity-weighted k-NN vote	taxonomy	tie-break when rules and votes disagree or confidence < 0.45
4	Risk Prediction	case-based KPI profile → fault-risk and performance models → risk and priority	ML models	—
5	Root Cause & Impact	KPI z-score deviations, ML driver ablation, prevalence of conditions in similar cases, SOP causes; users, SLA, MTTR, services, pattern prevalence	thresholds, models, retriever	writes the narrative and ranks causes
6	Recommendation	SOP-RAG plan; immediate / short-term / preventive phases; approval flags; escalation by priority	playbook, feedback action scores	grounded plan with citations
7	LLM-as-Judge	scores groundedness, diagnostic correctness, actionability, relevance and safety (1–5); verdict *pass* or *revise*	evidence summary	independent reviewer at temperature 0

Collaboration. If the judge returns *revise*, its issues go back to the recommendation agent, which re-plans (for example, it adds SOPs for symptoms that were not covered) and is judged again, at most once. This bounds cost and latency.

Case-based profile. For text-only queries, agent 4 builds a KPI profile from the median of the retrieved similar cases, overlaid with typical values for the symptoms in the query. This lets the ML models score a question that contains no numbers.

Why not LangGraph, CrewAI or AutoGen? The workflow is a fixed DAG with one feedback edge. Plain Python gives deterministic, testable control flow, no framework lock-in and lower latency. The agent interface maps one-to-one to LangGraph nodes if dynamic routing is needed later.

9. Predictive models

Model	Target	Features	Algorithm	Notes
Fault risk	Fault Occurrence Rate (%)	20 KPIs + season + weather	HistGradientBoostingRegressor	used for what-if analysis and ablation explanations
High-risk classifier	rate ≥ fleet P75	same	HistGradientBoostingClassifier	probability shown in the UI
Early warning	Fault Occurrence Rate (%)	without critical alarms / throughput	HGB regressor	honest leading-indicator forecast

Model	Target	Features	Algorithm	Notes
Performance	Throughput, Latency, Call Drop Rate	operating conditions only (targets, fault rate, critical alarms and auth failures excluded)	3 × HGB regressor	R^2 is reported alongside each forecast

HistGradientBoosting was chosen because it is fast on 100K rows, handles categoricals natively, needs little tuning and has no extra dependencies. **Explainability:** each prediction comes with local contributions computed by median-substitution ablation (f(x) – f(x with feature = fleet median)). This is model-agnostic and easy to explain to engineers. Global permutation importance is stored in `model_metrics.json`.

Model results (80/20 split, 20K test rows). High-risk classifier: ROC-AUC 0.9998, F1 0.9869, threshold 48.17 % fault rate.

model	R2	MAE	baseline MAE
Fault risk (full)	1.000	0.189	11.441
Fault risk (early warning)	0.641	6.880	11.441
Performance: Throughput (Mbps)	0.328	161.749	197.285
Performance: Latency (ms)	0.130	28.707	30.328
Performance: Call Drop Rate (%)	0.191	0.949	1.091

Top permutation importances (fault model): Critical Alarm Count (0.636), Authentication Failures (0.221), Throughput (Mbps) (0.157), Security Incidents (0.042), Successful Configuration Changes (%) (0.001), Weather (0.000). The performance models beat the mean baseline but have modest R^2 because the synthetic KPIs are largely independent; forecasts are shown with R^2 and an approximate interval.

10. Guardrails, security and compliance

- **Schema validation** (Pydantic): ranges for every KPI (e.g. signal –150…–20 dBm), enumerations for season, weather, category and priority, a 2-letter state format, a safe character set for city names, `top_k` ≤ 20 and batch size ≤ 500.
- **Input guardrails**: empty, short and long queries; control characters; 13 prompt-injection or jailbreak patterns (including SQL and script injection) → HTTP 422 `PROMPT_INJECTION`; a domain-relevance check → `OFF_TOPIC`.
- **Privacy**: emails, phone numbers, IMSIs and card numbers are redacted *before* any processing or LLM call, and the redaction is reported to the user. Only the sanitised query is logged.
- **Prompt hardening**: user text is delimited (`<<<…>>>`) and treated as data; the system prompt forbids inventing IDs, values or procedures.
- **Output guardrails**: hallucinated case IDs are removed, unsafe instructions are blocked (e.g. "disable all alarms"), and service-affecting steps (restart, rollback, lock, dispatch) are flagged `requires_approval`, in line with change-management practice.
- **Access**: an optional `X-API-Key` on all POST endpoints; request IDs for audit trails; secrets only via environment variables or `.env` (git-ignored).

11. Evaluation strategy

1. **Unit, API and agent tests** (pytest, 55+ cases): guardrails, parser, features, BM25, every endpoint, filters, the feedback loop, the LLM path with a scripted fake LLM (citation sanitising and safety), LLM-failure fallback, and the judge revision loop.
2. **Retrieval evaluation**: strategies (BM25, dense, hybrid, hybrid + re-rank, an alpha sweep) and embedding models (§5, §6).
3. **Recommendation evaluation** on 15 golden queries (including the three questions from the brief): category accuracy, symptom-extraction F1, SOP coverage, action-keyword recall against a reference answer, citation validity, judge scores and latency.

4 **DeepEval** (when an LLM key is configured): AnswerRelevancy, Faithfulness, ContextualRelevancy, GEval "Recommendation Quality" (against the reference) and GEval "Diagnosis Correctness" (against the retrieved context). The judge model is the configured provider, wrapped as a `DeepEvalBaseLLM`.

5 **Online validation**: the LLM-as-judge scores every `/analyze` response; engineers' ratings arrive through `/feedback`.

Recommendation evaluation results (15 golden queries, mode: deterministic/offline, 100000 cases indexed):

metric	value
latency_ms	543.053
category_correct	1.000
category_top1_strict	0.800
condition_f1	0.950
sop_coverage	1.000
action_keyword_recall	0.933
citation_validity	1.000
retrieval_symptom_match@k	0.996
judge_overall	4.912
judge_groundedness	5.000
judge_diagnostic_correctness	4.693
judge_actionability	5.000
judge_relevance	4.867
judge_safety	5.000
n_queries	15
n_cases_indexed	100000
p95_latency_ms	662.000

id	category	category_correct	condition_f1	sop_coverage	action_keyword_recall	judge_overall	latency_ms
G01	TRANSPORT_QOS	True	1.000	1.000	1.000	5.000	506.900
G02	EQUIPMENT_FAULT	True	0.400	1.000	1.000	4.960	423.200
G03	MOBILITY	True	1.000	1.000	1.000	5.000	544.400
G04	MOBILITY	True	1.000	1.000	1.000	5.000	430.500
G05	RAN_COVERAGE	True	1.000	1.000	1.000	5.000	400.600
G06	CAPACITY	True	1.000	1.000	1.000	5.000	875.500
G07	SECURITY	True	1.000	1.000	1.000	5.000	658.100
G08	CONFIGURATION	True	1.000	1.000	1.000	5.000	661.800
G09	EQUIPMENT_FAULT	True	0.857	1.000	1.000	4.600	662.000
G10	TRANSPORT_QOS	True	1.000	1.000	1.000	5.000	551.300
G11	ENVIRONMENT	True	1.000	1.000	0.750	4.600	531.200
G12	EQUIPMENT_FAULT	True	1.000	1.000	0.750	4.570	575.600
G13	TRANSPORT_QOS	True	1.000	1.000	1.000	5.000	470.000
G14	SECURITY	True	1.000	1.000	0.750	4.970	431.600
G15	TRANSPORT_QOS	True	1.000	1.000	0.750	4.980	423.100

12. Feedback loop (continuous improvement)

Every `/recommend` and `/analyze` response is logged with a `response_id`. Engineers rate it (1–5), can mark which cases helped and which actions worked, and can correct the category. These signals are used in three ways.

- **Retrieval**: a per-case net score, passed through tanh and capped at ±0.15, is added during re-ranking, so helpful precedents rise and misleading ones sink. `test_feedback_loop_changes_ranking` verifies this.
- **Recommendations**: per-category action effectiveness scores reorder the SOP steps.
- **Labels**: category corrections are collected as labelled examples for the evaluation set and for future retraining of the classifier.

13. Observability, scalability and deployment

- **Observability**: structured logs with request IDs and timings; an in-process metrics registry (per-endpoint count, p50/p95/max latency, 5xx count, guardrail blocks, PII redactions, judge revisions, feedback, LLM calls/failures/latency) at `/metrics`; alerts for p95 latency, error rate above 5 %, P1 bursts and guardrail spikes; a Streamlit monitoring tab. The registry interface maps directly to Prometheus or Azure Monitor exporters.
- **Performance** (100K cases, CPU laptop): hybrid retrieval about 40 ms (p50, in-process) and `/search` about 140 ms end to end; full deterministic `/analyze` about 0.5 s on average (§11 latency). With an LLM, latency is dominated by about 5 model calls (roughly 3–8 s).
- **Scaling path**: the API is stateless (artifacts are read-only), so it scales horizontally behind a load balancer.

For millions of records, move to FAISS IVF-PQ/HNSW or a managed vector store (Azure AI Search or Qdrant) with native filtering, move BM25 to OpenSearch/Elastic, keep feedback in Postgres, cache query embeddings and LLM outputs, and rebuild the index incrementally from streaming KPIs.

- **Deployment**: Dockerfile (artifacts built at image build time) and docker-compose (API + UI). On Azure: Container Apps or AKS, Blob Storage for artifacts, Key Vault for secrets, Azure OpenAI for the LLM.

14. Key trade-offs

Trade-off	Decision	What we gave up
Deterministic core vs LLM-centric	ML and rules for facts, the LLM for language	some fluency when running offline
Exact FAISS vs ANN	exact (100K)	memory grows linearly; needs ANN beyond about 5M vectors
Custom BM25 vs search engine	in-process scipy BM25	advanced analysers and distributed search
Data-driven percentile thresholds vs 3GPP absolute limits	percentiles (the dataset's scale differs from field norms, e.g. a mean loss of 6 %)	thresholds are relative to the fleet, not to engineering targets
Plain-Python orchestration vs framework	plain Python	built-in graph visualisation and checkpointing
One revision loop	at most 1	rare cases may still need a human
Synthetic dataset	used as-is, with findings documented	absolute model accuracy may not transfer to real networks

15. Limitations and future work

- Fault rate is deterministic in this dataset, so real-network accuracy needs retraining on live data (the pipeline supports it unchanged).
- Records have no timestamps or cell IDs, so true time-series forecasting and topology-aware root cause are not possible. Next steps: temporal models and alarm correlation graphs.
- Add a cross-encoder re-ranker, LLM query rewriting, Ragas metrics, A/B testing of ranking with feedback, and role-based access (Entra ID).